



AWS FROM DEVOPS PERSPECTIVE

STEVAN CVETKOVIĆ

JANUARY 2018

AGENDA

- INTRODUCTION
- AWS CLI
- IDENTITY AND ACCESS MANAGEMENT (IAM)
- ELASTIC COMPUTE CLOUD (EC2)
- VIRTUAL PRIVATE CLOUD (VPC)
- SIMPLE STORAGE SERVICE (S3)
- CODE COMMIT
- CODE PIPELINE
- CODE DEPLOY
- OPS WORKS
- USEFUL RESOURCES

INTRODUCTION

▪ Benefits of using Cloud

- Brings great amount of flexibility and speed with infrastructure provisioning
- Developers no longer need to wait on IT to setup infrastructure
- Great for quick testing your software on different Operating Systems and different environment setups
- Brings native support for declaring infrastructure as code
- Allows Dev and Ops teams to collaborate and learn together and from each other



INTRODUCTION

- Amazon launched **AWS** in 2006
- AWS operates in 16 geographical locations called **Regions**, which are spread over 3 continents (Europe, America and Asia) allowing faster content distribution to end-users and highly available services in case of outages or natural disasters impacting some of them
- Each Region has multiple **Availability Zones**, which consist of one or more discrete data centers, each with redundant power, networking and connectivity, housed in separate facilities



AWS CLI

- Powerful and easy to use Command Line Interface for AWS, available for all major platforms
- Enables you to interact with almost any existing AWS service from command line
- Gives you ability to query, create, configure and delete your AWS infrastructure and services without ever logging in to AWS Web interface
- Useful for scripting your frequent tasks, such as generating reports on the state of your infrastructure
- **AWS Access Key** and **AWS Secret Key** are required for authentication to AWS. You can get them and manage them (generate, enable/disable, delete) from **IAM**.
- You can also make use of AWS SDK if you need to build applications which will interact with AWS
- To get started with CLI, configure it first with: `$ aws configure`

IDENTITY AND ACCESS MANAGEMENT (IAM)

- **IAM** enables you to securely control access to AWS services and resources for your users
- Using IAM, you can create and manage AWS users and groups, and use permissions to allow and deny their access to AWS resources
- IAM is a feature of your AWS account offered at **no additional charge**. You will be charged only for use of other AWS services by your users, such as running EC2 instances and S3 storage in use.
- To use AWS CLI, you will first need to go to IAM on AWS Web interface to create your user account, assign it with appropriate permissions for your needs and get its AWS Access Key and AWS Secret Key

IDENTITY AND ACCESS MANAGEMENT (IAM)

- Best practices:
 - **Users** – Create individual users
 - **Groups** – Manage permissions with groups
 - **Permissions** – Grant least privilege
 - **Auditing** – Turn on AWS CloudTrail
 - **Password** – Configure a strong password policy
 - **MFA** – Enable MFA for privileged users
 - **Roles** – Use IAM roles for Amazon EC2 instances
 - **Sharing** – Use IAM roles to share access
 - **Rotate** – Rotate security credentials regularly
 - **Conditions** – Restrict privileged access further with conditions
 - **Root** – Reduce or remove use of root

IDENTITY AND ACCESS MANAGEMENT (IAM)

- To list all users: `$ aws iam list-users`
- To list all groups: `$ aws iam list-groups`

- To create a new user:

```
$ aws iam create-user --user-name endava
{
  "User": {
    "UserName": "endava",
    "Path": "/",
    "CreateDate": "2018-01-20T17:03:24.978Z",
    "UserId": "AIDAJWQ2PAKLXHBYYQSH2",
    "Arn": "arn:aws:iam::680381214567:user/endava"
  }
}
```

- Add user to “admin” group: `$ aws iam add-user-to-group --user-name endava --group-name admin`

IDENTITY AND ACCESS MANAGEMENT (IAM)

- Get info on user:

```
$ aws iam get-user --user-name endava
{
  "User": {
    "UserName": "endava",
    "Path": "/",
    "CreateDate": "2018-01-20T17:03:24Z",
    "UserId": "AIDAJWQ2PAKLXHBYYQSH2",
    "Arn": "arn:aws:iam::680381214567:user/endava"
  }
}
```

- List groups to which user belongs to:

```
$ aws iam list-groups-for-user --user-name endava
{
  "Groups": [
    {
      "Path": "/",
      "CreateDate": "2013-10-18T16:49:11Z",
      "GroupId": "AGPAIPHJSLDLU5UTXS",
      "Arn": "arn:aws:iam::680381214567:group/admin",
      "GroupName": "admin"
    }
  ]
}
```

IDENTITY AND ACCESS MANAGEMENT (IAM)

- Create Access and Secret key:

```
$ aws iam create-access-key --user-name endava
{
  "AccessKey": {
    "UserName": "endava",
    "Status": "Active",
    "CreateDate": "2018-01-20T17:06:13.883Z",
    "SecretAccessKey": "X8sWC75uV33Qf0pasd1eFi91siQPZM0Vedo5m0bM",
    "AccessKeyId": "AIDAJWQ2PAKLXHBYQQSH2"
  }
}
```

- To delete a user:

1. \$ aws iam remove-user-from-group --user-name endava --group-name admin
2. \$ aws iam delete-access-key --user-name endava --access-key-id AIDAJWQ2PAKLXHBYQQSH2
3. \$ aws iam delete-user --user-name endava

ELASTIC COMPUTE CLOUD (EC2)

- **EC2** is a web service that provides you with complete control of your computing resources, such as VMs that you are launching on it and all configurations related to them
- EC2 reduces the time required to obtain and boot new server instances to minutes, allowing you to quickly scale capacity, both up and down, as your computing requirements change
- You are charged only for capacity that you actually use
- EC2 offers a number of OS images you can use to launch your machines. Those images are called **AMIs (Amazon Machine Image)** and are identified by their ID, such as `ami-4fffc834`, which is different in every AWS Region for the same OS

ELASTIC COMPUTE CLOUD (EC2)

1. Create a new SSH keypair (public key will be on AWS) for reaching your instances from your PC:

```
$ aws ec2 create-key-pair --key-name stevan
```

2. Create a new Security Group:

```
$ aws ec2 create-security-group --group-name stevan-test-sg --description "My SG"
```

3. Allow inbound access to port 22 (SSH) only from our own IP:

```
$ aws ec2 authorize-security-group-ingress --group-name stevan-test-sg --protocol tcp \  
--port 22 --cidr <your-IP>/32
```

4. Create a new EC2 instance using above SG:

```
$ aws ec2 run-instances --image-id ami-4fffc834 --count 1 --instance-type c3.large \  
--key-name stevan --security-groups stevan-test-sg
```

ELASTIC COMPUTE CLOUD (EC2)

- Get details on our EC2 instance status and properties, including its IP:

```
$ aws ec2 describe-instance-status --instance-id i-08c361d82f54a8820
```

```
$ aws ec2 describe-instances --instance-id i-08c361d82f54a8820
```

- Shutdown and permanently remove your instance:

```
$ aws ec2 terminate-instances --instance-id i-08c361d82f54a8820
```

VIRTUAL PRIVATE CLOUD (VPC)

- **VPC** lets you provision a logically isolated section of the cloud where you can launch AWS resources in a virtual network that you define
- Offers complete control over your virtual networking environment, including selection of your own IP address range, creation of **subnets**, and configuration of **route tables** and **network gateways**
- You can leverage multiple layers of security, including **security groups** and **network access control lists**, to help control access to Amazon EC2 instances in each subnet
- Additionally, you can create a Hardware Virtual Private Network (VPN) connection between your corporate data center and your VPC and leverage the cloud as an extension of your corporate data center

VIRTUAL PRIVATE CLOUD (VPC)

1. Create new VPC: `$ aws ec2 create-vpc --cidr-block 10.0.0.0/16`

2. Create subnet under the new VPC:

```
$ aws ec2 create-subnet --vpc-id vpc-0d868875 --cidr-block 10.0.1.0/24
```

3. Optionally, we can change subnet settings to enforce public IP assignment on EC2 instance launch:

```
$ aws ec2 modify-subnet-attribute --subnet-id subnet-204f1444 --map-public-ip-on-launch
```

4. Create SG in VPC:

```
$ aws ec2 create-security-group --description "My SG" --group-name stevan-test-sg-vpc  
--vpc-id vpc-0d868875
```

5. Authorize VPC SG for SSH inbound access from specific source IP:

```
$ aws ec2 authorize-security-group-ingress --group-id sg-f8dcef8c --protocol tcp --port 22  
--cidr <your-IP>/32
```

6. Let's also provision our instance on creation by installing Docker. Content of *aws-ec2-user-data.sh* file:

```
#!/bin/bash  
yum update -q -y && yum install docker -q -y  
/etc/init.d/docker start  
gpsswd -a ec2-user docker
```

VIRTUAL PRIVATE CLOUD (VPC)

7. Create and provision EC2 instance in VPC:

```
$ aws ec2 run-instances \
  --image-id ami-4fffc834 \
  --count 1 \
  --instance-type m4.large \
  --block-device-mappings
' [{"DeviceName": "/dev/xvda", "Ebs": {"VolumeSize": 40, "DeleteOnTermination": true, "VolumeType": "gp2"
  "}}} ]' \
  --key-name stevan \
  --security-group-ids sg-f8dcef8c \
  --subnet-id subnet-204f1444 \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value='MyInstance'}]' \
  --user-data file://aws-ec2-user-data.sh
```

VIRTUAL PRIVATE CLOUD (VPC)

- If you now try to SSH to your new EC2 instance from your PC, **it won't work!** You need to do a few additional actions to make VPC accessible from the Internet:

1. Create Internet Gateway and attach it to your VPC:

```
$ aws ec2 create-internet-gateway
```

```
$ aws ec2 attach-internet-gateway --internet-gateway-id igw-2e192157 --vpc-id vpc-0d868875
```

2. Find a route table ID associated to your VPC:

```
$ aws ec2 describe-route-tables --filters "Name=vpc-id,Values=vpc-0d868875"
```

3. Add new route entry into the route table to allow access to your VPC from Internet (0.0.0.0/0):

```
$ aws ec2 create-route --route-table-id rtb-36bdf84b --destination-cidr-block 0.0.0.0/0 --gateway-id igw-2e192157
```

SIMPLE STORAGE SERVICE (S3)

- **S3** is object storage built to store and retrieve any amount of data from anywhere – web sites and mobile apps, corporate applications, and data from IoT sensors or devices
- It is designed to deliver 99.999999999% durability and provides comprehensive security and compliance capabilities
- AWS charges you based on size of the data you are storing
- If you are rarely accessing some of your data (old backups, archived data, etc.) you can move it to much cheaper, but slower to restore from, **Glacier** and cut your expenses

SIMPLE STORAGE SERVICE (S3)

- Create a new S3 bucket:

```
$ aws s3api create-bucket --bucket stevan-test
```

- List bucket content:

```
$ aws s3 ls s3://stevan-test/
```

- Copy file to the bucket:

```
$ aws s3 cp test.txt s3://stevan-test/
```

- Delete file from the bucket:

```
$ aws s3 rm s3://stevan-test/test.txt
```

- Delete the bucket:

```
$ aws s3api delete-bucket --bucket stevan-test
```

CODE COMMIT

- **CodeCommit** is a fully-managed source control service that makes it easy to host secure and highly scalable private Git repositories
- It eliminates the need to operate your own source control system or worry about scaling its infrastructure
- You can use it to store anything from source code to binaries, and it works seamlessly with Git

CODE COMMIT

- To get started, upload your SSH public key to your AWS account, if not already:

```
$ aws iam upload-ssh-public-key --user-name stevan \  
    --ssh-public-key-body ""`cat ~/.ssh/id_rsa.pub`""
```

- Then, you can create a new Git repository on CodeCommit:

```
$ aws codecommit create-repository --repository-name stevan-test \  
    --repository-description "My repo"
```

- Clone your new repo with the given SSH URL from previous command and add also **SSHPublicKeyId** value from the 1st command so to indicate which SSH public key you are using:

```
$ git clone ssh://APKPOFXVBFC7QU2WQ@git-codecommit.us-east-1.amazonaws.com/v1/repos/steven-test
```

CODE PIPELINE

- **CodePipeline** is a continuous integration and continuous delivery service for fast and reliable application and infrastructure updates
- CodePipeline builds, tests, and deploys your code every time there is a code change, based on the release process models you define.
- You can easily build out an end-to-end solution by using pre-built plugins for popular third-party services like GitHub or integrate your own custom plugins into any stage of your release process
- You only pay for resources you use through it, like EC2 instances

CODE DEPLOY

- **CodeDeploy** is a service that automates software deployments to a variety of compute services including Amazon EC2, AWS Lambda, and instances running on-premises
- It makes it easier to rapidly release new features, helps avoid downtime during application deployment, and handles the complexity of updating your applications
- You can use it to automate software deployments, eliminating the need for error-prone manual operations
- The service scales to match your deployment needs, from a single Lambda function to thousands of EC2 instances

OPS WORKS

- **OpsWorks** is a configuration management service that provides managed instances of Chef and Puppet.
- OpsWorks lets you use Chef and Puppet to automate how servers are configured, deployed, and managed across your Amazon EC2 instances or on-premises compute environments.
- It has three offerings:
 - OpsWorks for Chef Automate (fully managed enterprise Chef server)
 - OpsWorks for Puppet Enterprise (fully managed enterprise Puppet)
 - OpsWorks Stacks (application modeling and management with Chef Solo)

OPS WORKS

- Getting Started with **OpsWorks Stacks**:

1. Create a new **Stack**. Stack holds the complete environment you need to host your application. Make sure to enable “Use custom Chef cookbooks” option and point it to your cookbooks (from Git, S3 or HTTP).
2. Add a new **Layer**. A layer is a blueprint for a set of Amazon EC2 instances. It specifies the instance's settings, associated resources, installed packages, profiles, and security groups. You can also Configure your Layer. Add names of appropriate Chef recipes to lifecycle events of your instances, for example: to set up and configure your instances and deploy your app.
3. Create and start an EC2 **instance** (from OpsWorks) belonging to your Layer
4. Create an **App**. An app represents code stored in a repository that you want to install on application server instances.
5. **Deploy** your App onto given instance(s) in your Layer

USEFUL RESOURCES

- **Official AWS documentation:** <https://aws.amazon.com/documentation/>
- **AWS free tier:** <https://aws.amazon.com/free/>

THANK YOU

STEVAN CVETKOVIĆ
SENIOR DEVOPS ENGINEER

stevan.cvetkovic@endava.com